

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский
Нижегородский государственный университет им. Н.И. Лобачевского»

Институт экономики
Отделение среднего профессионального образования

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №4

Выполнил студент группы
3525П1ИН1 специальность СПО
09.02.07 Информационные системы
и программирование

_____ Е.А.Пчеляков
(подпись)

Научный руководитель:
преподаватель ИНЭК СПО

_____ В.А.Киселева
(подпись)

Нижний Новгород
2026

Введение

В рамках данного отчета проведен анализ программного кода функции `'calc_bonus'`, предназначенной для автоматического расчета размера бонуса сотрудников. Цель анализа — выявление логики начисления, оценка корректности алгоритма и выявление потенциальных проблем производительности. Краткий вывод: алгоритм корректно выполняет математические операции, однако использует неоптимальный подход с циклом `'for'` для повторяющихся вычислений, что может замедлить работу системы при больших объемах данных.

Контекст и ограничения

Текущее состояние (As-Is): Имеется готовая функция на языке Python, которая рассчитывает бонус в зависимости от трех параметров: суммы (`'x'`), количества (`'y'`) и категории (`'z'`). Логика имеет два больших ответвления в зависимости от суммы `'x'` (больше или меньше 1000).

Целевое состояние (To-Be): Функция должна не только считать бонус, но и делать это максимально быстро, без лишней нагрузки на процессор.

Ограничения: Код написан на Python. Мы не можем менять входные параметры (`x`, `y`, `z`), но можем проанализировать внутреннюю логику.

Методология анализа

Для анализа были применены следующие методы:

Статический анализ кода: построчное чтение и понимание логики ветвлений (`'if/else'`) и циклов (`'for'`).

Оценка сложности алгоритма: проверка на наличие избыточных вычислений.

Построение блок-схемы: визуализация алгоритма для наглядного представления путей выполнения.

Профиль продукта

Название: `calc\_bonus`.

Входные данные: `x` (сумма), `y` (количество), `z` (категория).

Логика:

Если $x > 1000$: запускается цикл `y` раз. Внутри цикла выбирается процент (15%, 10% или 5%) в зависимости от `z`. Результат накапливается.

Если $x \leq 1000$: используется простое умножение. Процент (2% или 1%) зависит от того, больше ли `y` десяти.

Возможности: Функция универсальна и покрывает широкий спектр сценариев начисления премий.

Результаты сравнения (Анализ эффективности)

В ходе анализа выявлены следующие особенности:

Плюс: Логика покрывает все возможные случаи (большие/маленькие суммы, разные категории).

Минус (Критический): Использование цикла `for i in range(y)` для простого сложения одинаковых чисел (когда $x > 1000$) является ****избыточным****. Математически это равносильно умножению. Если переменная `y` будет очень большой, время выполнения программы увеличится линейно, что неэффективно.

Анализ рисков внедрения

Риск производительности: При больших значениях `y` (например, если это количество дней в году или большое число сделок) цикл может вызвать задержку в расчетах.

Риск читаемости: Вложенные условия (`if` внутри `if` внутри `for`) делают код сложным для быстрого понимания новыми разработчиками.

Митигация (решение): Заменить цикл на математическую формулу $res = x$

* процент * y`.

Заключение и рекомендации

Финальный выбор: Алгоритм признан рабочим, но требующим рефакторинга.

Обоснование: Текущий код выполняет задачу, но не оптимален.

План перехода (Roadmap):

Шаг 1: Заменить цикл `for` на умножение в блоке `if x > 1000`.

Шаг 2: Вынести магические числа (0.15, 0.10, 0.05) в константы для удобства.

Альтернативный вариант (Plan B): Оставить код как есть, если входные данные `y` гарантированно малы (меньше 100), и производительность не критична.